

Reviewer Pack — Spectral Gap Conjecture for Tseitin Tensor Product Complexit...

Entry 381a7626353c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 22:03:07 UTC

Spectral Gap Conjecture for Tseitin Tensor Product Complexity

Entry ID: 381a7626353c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 22:03:07 UTC

1. Conjecture

Field A (mathematical branch): Quantum Walks and Spectral Graph Theory **Field B** (complexity object): Tseitin Tensor Product Complexity

Statement:

For a given 3-CNF formula φ , the spectral gap of the adjacency matrix of the corresponding Tseitin graph is $\Theta(f(n))$, where $f(n) = \log(n)$. Specifically, there exists a constant C such that for all 3-CNF formulas with n variables, $|\lambda_{\min}(A(\varphi)) - (1/2)| \leq C/n$, where $\lambda_{\min}(A(\varphi))$ is the smallest eigenvalue of the adjacency matrix $A(\varphi)$.

Rationale (proposer's reasoning):

Quantum walks on Tseitin graphs have been used to study communication complexity and circuit lower bounds. The spectral gap of a graph's adjacency matrix provides insights into the efficiency of quantum algorithms, potentially revealing structural properties of Tseitin tensor products that are related to their computational complexity.

Taxonomy category: SPECTRAL_GRAPH_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a3adee622d92ba0b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a 3-CNF formula φ , the spectral gap of its Tseitin graph's adjacency matrix is supported if $|\lambda_{\min}(A(\varphi)) - (1/2)| \leq C/n$ with $C = 10$ and falsified if any seed produces $|\lambda_{\min}(A(\varphi)) - (1/2)| > 0.5$ for $n \in \{20, 30, 40\}$

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Tseitin Tensor Product Complexity" AND "spectral gap" AND quantum walks - "3-CNF formula" AND Tseitin graph AND spectral eigenvalues" - eigenvalue gap in Tseitin graphs for 3-CNF complexity

Top relevant hits considered: - [http://arxiv.org/abs/math/0611649v2] The Distribution of the Largest Non-trivial Eigenvalues in Families of Random Regular Graphs - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/math/0602191v4] On the maximum number of cliques in a graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = random.sample(['x' + str(i), 'y' + str(i)], 2) + ['z' + str(random.randint(1, n))]
            clauses.append(clause)
        return clauses

    def tseitin_graph(clauses):
        # Construct Tseitin graph here
        pass

    def adjacency_matrix(graph):
        # Compute adjacency matrix here
        pass

    def smallest_eigenvalue(matrix):
        # Compute smallest eigenvalue here
        pass

    n_values = [20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(10): # Test each size with 10 instances
```

```

        phi = generate_3cnf(n)
        G_phi = tseitin_graph(phi)
        A_phi = adjacency_matrix(G_phi)
        lambda_min = smallest_eigenvalue(A_phi)
        results.append(abs(lambda_min - 0.5) / (n / 10))

metric_value = sum(results) / len(results)
conjecture_holds = all(x <= 0.5 for x in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "spectral_gap",
    "metric_value": metric_value,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = (sum((x - mean) ** 2 for x in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r <= 0.5) / len(results)

    if all(r <= 0.5 for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(r > 0.5 for r in results):
        first_failing_seed = seeds[results.index(max(results))]
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b36851a2.py", line 70, in <module>
 result = run_trial(seed)
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_b36851a2.py", line 45, in run\_trial  
 phi = generate\_3cnf(n)  
 ~~~~~

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b36851a2.py", line 24, in generate_3cnf
    clause = random.sample(['x' + str(i), 'y' + str(i)], 2) + ['z' + str(random.randint(1, n))]
                                ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented it from producing any data or results. | next: Review the test code for the definition of 'i' and ensure that all necessary variables are defined before use.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22479
2	preregistration	ollama_remote	glm4:latest	0	0	9793
3	novelty	ollama_remote	glm4:latest	0	0	8444
4	novelty	ollama_remote	glm4:latest	0	0	16718
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17554
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30343
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15570
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8604
9	judge	ollama_remote	glm4:latest	0	0	8801

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 138305 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/381a7626353c.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/381a7626353c.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/381a7626353c.tar.gz* (if generated)